Materia: Programación IV

UNIDAD 6 · RELACIONES Y ESTADO

TRABAJO PRÁCTICO

CRUD Categorías + CRUD Productos

Revisor de la unidad: Prof. Juan Ignacio Sarmiento

Objetivo general

Desarrollar una aplicación Full Stack conectando un frontend React + TypeScript con un backend FastAPI. El estudiante implementa el CRUD completo de Categorías desde la interfaz de usuario, integrando el backend mediante fetch nativo. Los CRUDs de Producto y ProductoCategoria se construyen en el backend y se demuestran mediante un archivo REST Client.

Marco teórico

Concepto	Aplicación en el proyecto
React	Biblioteca para construir interfaces de usuario mediante componentes reutilizables.
TypeScript	Superset tipado de JavaScript para detectar errores en compilación y mejorar la legibilidad.
Vite	Herramienta de build moderna que inicializa proyectos React con TypeScript, reemplazando Create React App.
FastAPI	Framework Python de alto rendimiento para construir APIs REST. Permite definir endpoints, validar datos con Pydantic y exponer documentación automática en /docs.
Pydantic	Librería de validación de datos usada por FastAPI para definir modelos (schemas) con tipado estricto.
fetch nativo	API del navegador para realizar peticiones HTTP. Se usa con async/await para consumir la API REST desde el frontend.
useState	Hook de React para manejar estado local en un componente funcional (ej: lista de categorías, estado del modal).
useEffect	Hook de React para ejecutar efectos secundarios (ej: fetch al montar el componente para cargar datos).
Tipado estático	Construcción de TypeScript (`type` o `interface`) que define la forma de un objeto (ej: Categoria, Producto).

Concepto	Aplicación en el proyecto
Tailwind CSS	Framework de CSS utilitario. Estilos aplicados directamente en el JSX mediante clases predefinidas.
CORS	Mecanismo que permite al frontend (localhost:5173) comunicarse con el backend (localhost:8000) en dominios distintos.

Caso práctico

Parte A: Backend con FastAPI

Estructura esperada del proyecto:

```
fastapi-productos/
│
├── app/
│   ├── __init__.py
│   ├── main.py
│   │
│   ├── core/
│   │   ├── __init__.py
│   │   └── database.py
│   │
│   ├── categoria/
│   │   ├── __init__.py
│   │   ├── router.py
│   │   ├── model.py
│   │   ├── schema.py
│   │   └── service.py
│   │
│   ├── producto/
│       ├── __init__.py
│       ├── router.py
│       ├── model.py
│       ├── schema.py
│       └── service.py
├── .env
├── .gitignore
├── requirements.txt
└── README.md
```

a) Crear y activar un entorno virtual Python, crear el archivo de dependencias e instalar las dependencias. El archivo de dependencias incluye `fastapi[standard]`, que trae el comando `fastapi dev` del paso e).

```
python -m venv .venv
source .venv/bin/activate   # macOS/Linux
.venv\Scripts\activate      # Windows
pip install -r requirements.txt
```

b) Definir los models y schemas con SQLModel. Los datos se persisten en una base de datos, no en una lista en memoria.

Categoria	Producto	Producto_Categoria
id: int	id:int	producto_id:int
nombre: str	nombre:str	categoria_id:int
descripcion: str	descripcion:str	
	precio_base:str	
	imagen_url:list[]	
	disponible: bool	

c) Implementar los endpoints de Categorías en `app/categoria/router.py` y habilitar CORS en `app/main.py` para poder consumir la API desde el Front-End (código de ejemplo):

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:5173'],
    allow_methods=['*'],
    allow_headers=['*'],
)
```

d) Implementar también los endpoints de Producto. Estos no necesitan conectarse al frontend: se demuestran con REST Client. El archivo .http se entrega junto con el proyecto del backend.

e) Ejecutar el servidor desde la carpeta `fastapi-productos/` :

```
python -m fastapi dev app/main.py
```

Parte B: Frontend con React + TypeScript

a) Inicializar el proyecto:

```
pnpm create vite@latest tp-categorias --template react-ts
cd tp-categorias
pnpm install
```

b) Instalar y configurar Tailwind CSS.

c) Definir el tipo en `src/types/categoria.ts`:

```
export type Categoria = {
  id: number;
  nombre: string;
  descripcion: string;
};
```

d) Crear los componentes en `src/components/` (.tsx):

- **Navbar.tsx**: barra de navegación con el nombre de la app.
- **CategoriaCard.tsx**: tarjeta que muestra una categoría (nombre + descripción). Recibe props tipados con el tipo Categoria. Incluye botones Editar y Eliminar.
- **CategoriaList.tsx**: recibe `Categoria[]` por props y renderiza las tarjetas.
- **CategoriaModal.tsx**: formulario de alta/edición dentro de un modal. Se abre y cierra con useState. Llama a onSubmit al guardar.

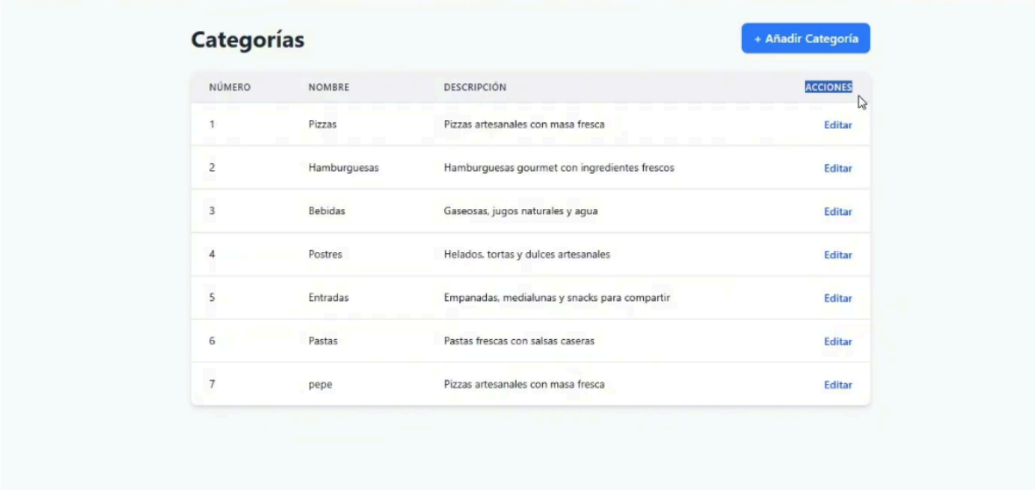
e) En App.tsx implementar la lógica principal:

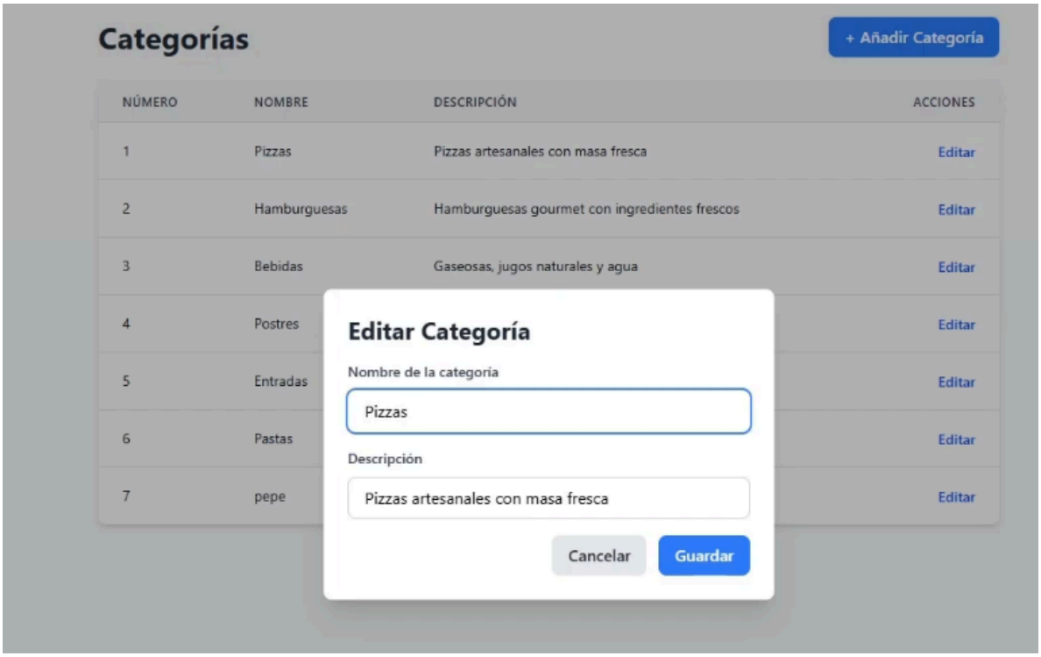
- useState para la lista de categorías, el estado del modal (abierto/cerrado) y la categoría seleccionada para edición.
- useEffect para cargar las categorías al montar el componente (GET /categorias).
- Funciones handleCreate, handleUpdate, handleDelete que usen fetch nativo hacia el backend.
- Pasar las funciones y el estado como props a los componentes hijos.

f) Verificar que el proyecto compila sin errores de TypeScript con `pnpm build` (ejecuta `tsc -b` antes de empaquetar; `pnpm dev` levanta el servidor de desarrollo pero no verifica los tipos) y correrlo con `pnpm dev`:

```
pnpm build
pnpm dev
```

Ejemplo de pantalla Front-End





Conclusiones esperadas

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Integración Full Stack:** El frontend React consume la API FastAPI mediante fetch nativo. El CRUD de Categorías funciona de extremo a extremo.
- **Modal con useState:** El formulario de alta y edición está dentro de un modal controlado con estado booleano.
- **Hooks aplicados correctamente:** useState y useEffect utilizados en App.tsx. Los demás componentes son funcionales puros, con una excepción: el formulario de CategoriaModal puede usar el custom hook `useForm` visto en la Actividad 4.

- **Tipado TypeScript:** Tipos Categoria y Producto definidos. Props tipados en todos los componentes .tsx.
- **Backend REST documentado:** Endpoints de Producto y ProductoCategoria funcionando y demostrados en el archivo .http.
- **CORS configurado:** El backend permite peticiones desde el origen del frontend.

Rúbrica de evaluación

Criterio	Descripción	Puntos
CRUD Categorías – Frontend	Listar, crear, editar y eliminar categorías desde React usando fetch nativo y useState/useEffect.	30
Modal funcional	Modal que se abre para crear y editar categorías, controlado con useState. Se cierra al guardar o cancelar.	15
CRUD Categorías – Backend	Endpoints FastAPI GET /categorias, POST, PUT/{id}, DELETE/{id} con Pydantic y persistencia en base de datos.	20
CRUD Productos y relación – REST Client	Endpoints de Productos y ProductoCategoria demostrados con REST Client (.http). No requiere UI.	15
Tipado TypeScript	Tipos Categoria y Producto tipados. Props tipados en todos los componentes.	10
Estructura y buenas prácticas	Carpetas components/ y types/. Tailwind CSS aplicado. Sin errores de TS en consola.	10

Modalidad de entrega

Se entregan los dos proyectos de una de estas dos formas: **dos archivos .zip por separado** (uno por proyecto) o **un único .zip** con las carpetas `frontend/` y `backend/`. En cualquiera de los dos casos:

1. **Proyecto Frontend**: sin la carpeta node_modules.
2. **Proyecto Backend**: con el archivo .http de REST Client.

Recursos adicionales

Documentación oficial:

- React: https://react.dev/
- TypeScript: https://www.typescriptlang.org/docs/
- Tailwind CSS: https://tailwindcss.com/docs/
- Vite: https://vitejs.dev/

- FastAPI: https://fastapi.tiangolo.com/